

CS111

Introduction to Computing Science



Chapter Six: Arrays and Vectors

Chapter Goals

- To become familiar with using arrays and vectors to collect values
- To learn about common algorithms for processing arrays and vectors
- To write functions that receive and return arrays and vectors
- To be able to use two-dimensional arrays

Using Arrays



Mail, mail and more mail – how to manage it?

Using Arrays

- Arrays are **data structures**.
 - They are used to store a group of objects/ variables.
- All elements of an array must be of the **same data type**
 - type: *int, float, char, double*.
- The elements are stored **sequentially in memory**
 - This allows for 'easy' access.
 - It also allow for easy mistakes.

Using Arrays

In arrays,
the stored data is of
the *same* type

Using Arrays

Think of a sequence of data:

32 54 67.5 29 35 80 115 44.5 100 65

(all of the same type, of course)
(storable as **doubles**)

Using Arrays

32 54 67.5 29 35 80 115 44.5 100 65

Which is the largest in this set?

(You must look at every single value to decide.)

Using Arrays

32 54 67.5 29 35 80 115 44.5 100 65

So you would create a variable for each,
of course!

```
double n1, n2, n3, n4, n5, n6, n7, n8, n9, n10;
```

Then what ???

Using Arrays

- Try doing it for 3 doubles : $n1$, $n2$, $n3$

```
double min_sofar = n1;  
if (n2 < min_sofar)  
    min_sofar = n2;  
if (n3 < min_sofar)  
    min_sofar = n3;
```

- Notice that a separate if statement is needed for each object in the list because, although the objects names are similar, each object is totally independent of the other objects.
- You cannot loop through variable names $n1, n2, \dots$

Using Arrays

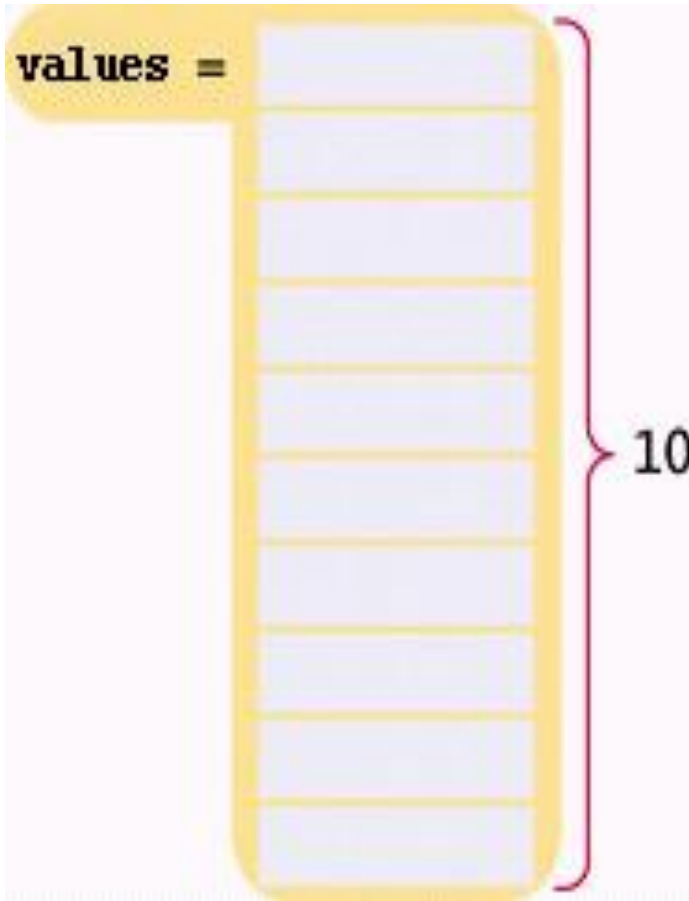
That would have been difficult with ten separate variables!

```
double n1, n2, n3, n4, n5, n6, n7, n8, n9, n10;
```

And what if there needed to be another double in the set?

ARGH!

Defining Arrays



An “array of double”

Ten elements of **double** type
can be stored under one name
as an array.

`double values[10];`

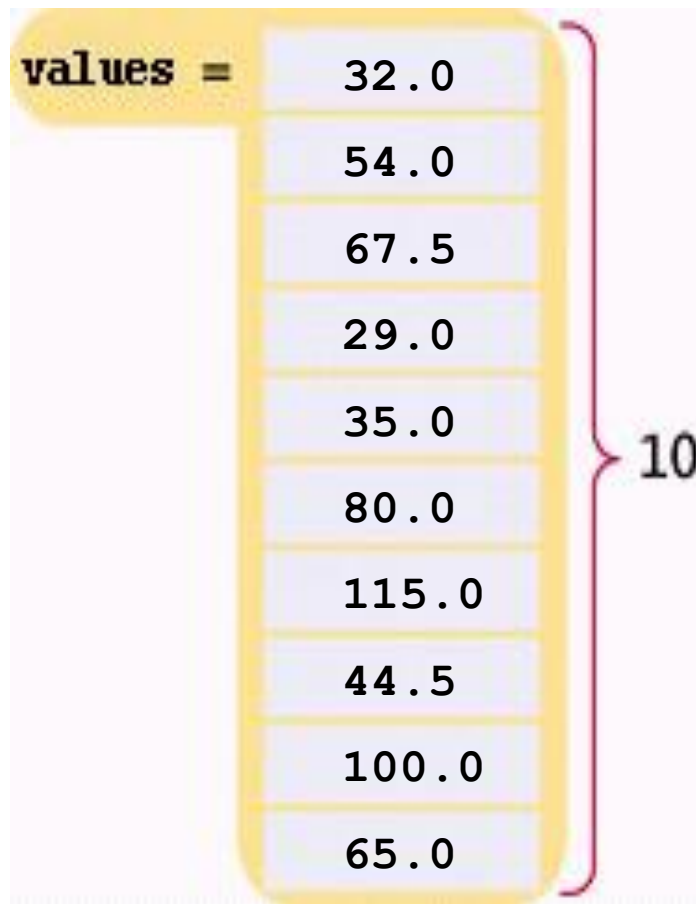
type of each element

quantity of elements – the “size” of the array,
must be a constant

Defining Arrays with Initialization

When you define an array, you can specify the initial values:

```
double values[] = { 32, 54, 67.5, 29, 35, 80, 115, 44.5, 100, 65 };
```



Array Syntax

Defining an Array

Element type Name Size

```
double values[5] = { 32, 54, 67.5, 29, 34.5 };
```

Size must be a constant.

Ok to omit size if initial values are given.

Use brackets to access an element.

```
values[i] = 0;
```

Optional list of initial values

The index must be ≥ 0 and $<$ the size of the array.

Array Syntax

Table 1 Defining Arrays

<code>int numbers[10];</code>	An array of ten integers.
<code>const int SIZE = 10; int numbers[SIZE];</code>	It is a good idea to use a named constant for the size.
 <code>int size = 10; int numbers[size];</code>	Caution: In standard C++, the size must be a constant. This array definition will not work with all compilers.
<code>int squares[5] = { 0, 1, 4, 9, 16 };</code>	An array of five integers, with initial values.
<code>int squares[] = { 0, 1, 4, 9, 16 };</code>	You can omit the array size if you supply initial values. The size is set to the number of initial values.
<code>int squares[5] = { 0, 1, 4 };</code>	If you supply fewer initial values than the size, the remaining values are set to 0. This array contains 0, 1, 4, 0, 0.
<code>string names[3];</code>	An array of three strings.

Accessing an Array Element

An array element can be used like any variable.

To access an array element, you use the notation:

values[i]

where **i** is the *index*.

Accessing an Array Element



Put the junk mail in there

in `mailboxes[356]`

Accessing an Array Element

To access the element at index 4 using this notation: **values[4]**

4 is the *index*.

values =

32.0

54.0

67.5

29.0

35.0

80.0

115.0

44.5

100.0

65.0

10

```
double values[10];
```

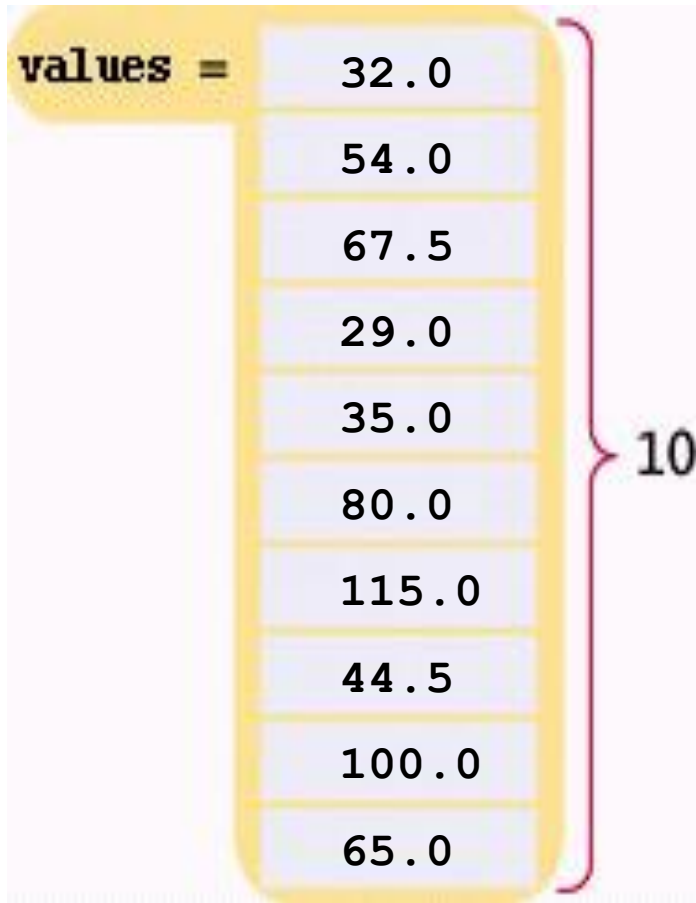
```
...
```

```
cout << values[4] << endl;
```

The output will be 35.0.

Accessing an Array Element

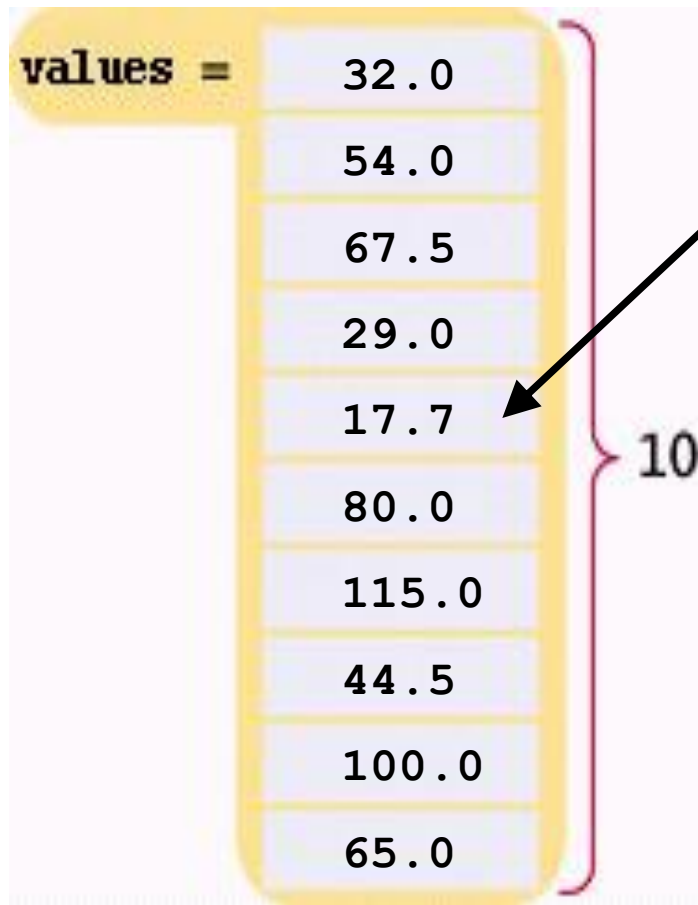
The same notation can be used to change the element.



```
values[4] = 17.7;
```

Accessing an Array Element

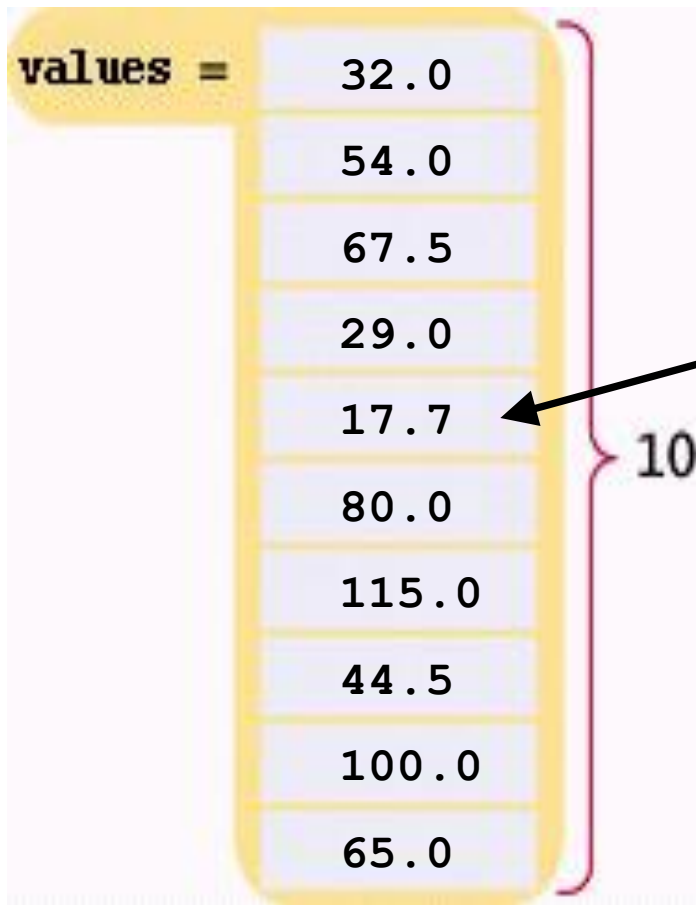
The same notation can be used to change the element.



`values[4] = 17.7;`

Accessing an Array Element

The same notation can be used to change the element.



```
values[4] = 17.7;  
cout << values[4] << endl;
```

The output will be 17.7.

Accessing an Array Element

You might have thought those last two slides were wrong: **values[4]** is getting the data from the “fifth” element.

values =	32.0	[0]
	54.0	[1]
	67.5	[2]
	29.0	[3]
	17.7	[4]
	80.0	[5]
	115.0	[6]
	44.5	[7]
	100.0	[8]
	65.0	[9]

```
cout << values[4] << endl;
```

In C++ and most computer languages, indexing starts with 0.

Accessing an Array Element

That is, the legal elements for the **values** array are:

values[0], the ***first*** element

values[1], the second element

values[2], the third element

values[3], the fourth element

values[4], the fifth element

...

values[9], the tenth ***and last legal*** element

recall: **double values**[10];

The index must be ≥ 0 and ≤ 9 .

0, 1, 2, 3, 4, 5, 6, 7, 8, 9 is 10 numbers.

Partially-Filled Arrays

Suppose an array can hold 10 elements:

Does it always hold 10?

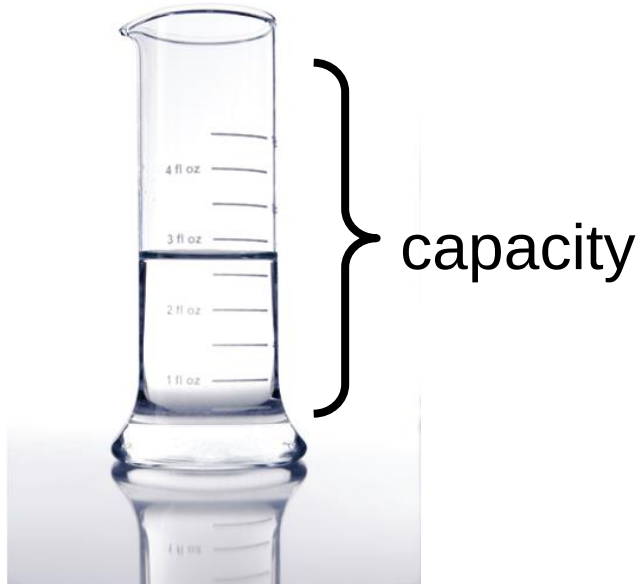
Guess not!



Partially-Filled Arrays – Capacity

How many elements, at most, can an array hold?

We call this quantity the *capacity*.



Partially-Filled Arrays – Capacity

For example, we may decide for a particular problem that there are usually ten or 11 values, but never more than 100.

We would set the capacity with a `const`:

Remember: The size of an array has to be constant.

```
const int CAPACITY = 100;  
double values[CAPACITY];
```

Partially-Filled Arrays

Arrays will usually hold less than CAPACITY elements.

We call this kind of array a *partially filled array*:

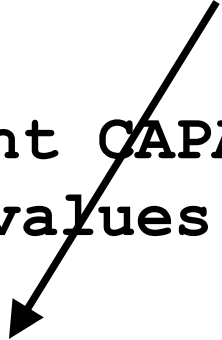


Partially-Filled Arrays –Variable for Size

But how many actual elements are there in a partially filled array?

We will use a *companion variable* to hold that amount:

```
const int CAPACITY = 100;  
double values[CAPACITY];  
  
int current_size = 0; // array is empty
```

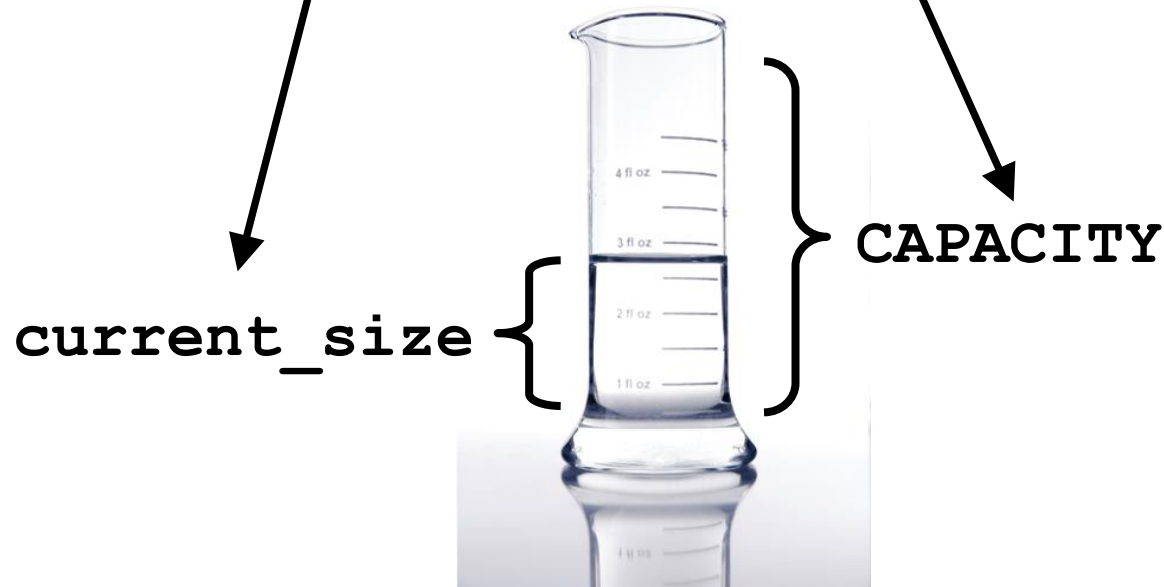


Suppose we add four elements to the array?

Partially-Filled Arrays –Variable for Size

```
const int CAPACITY = 100;  
double values[CAPACITY];
```

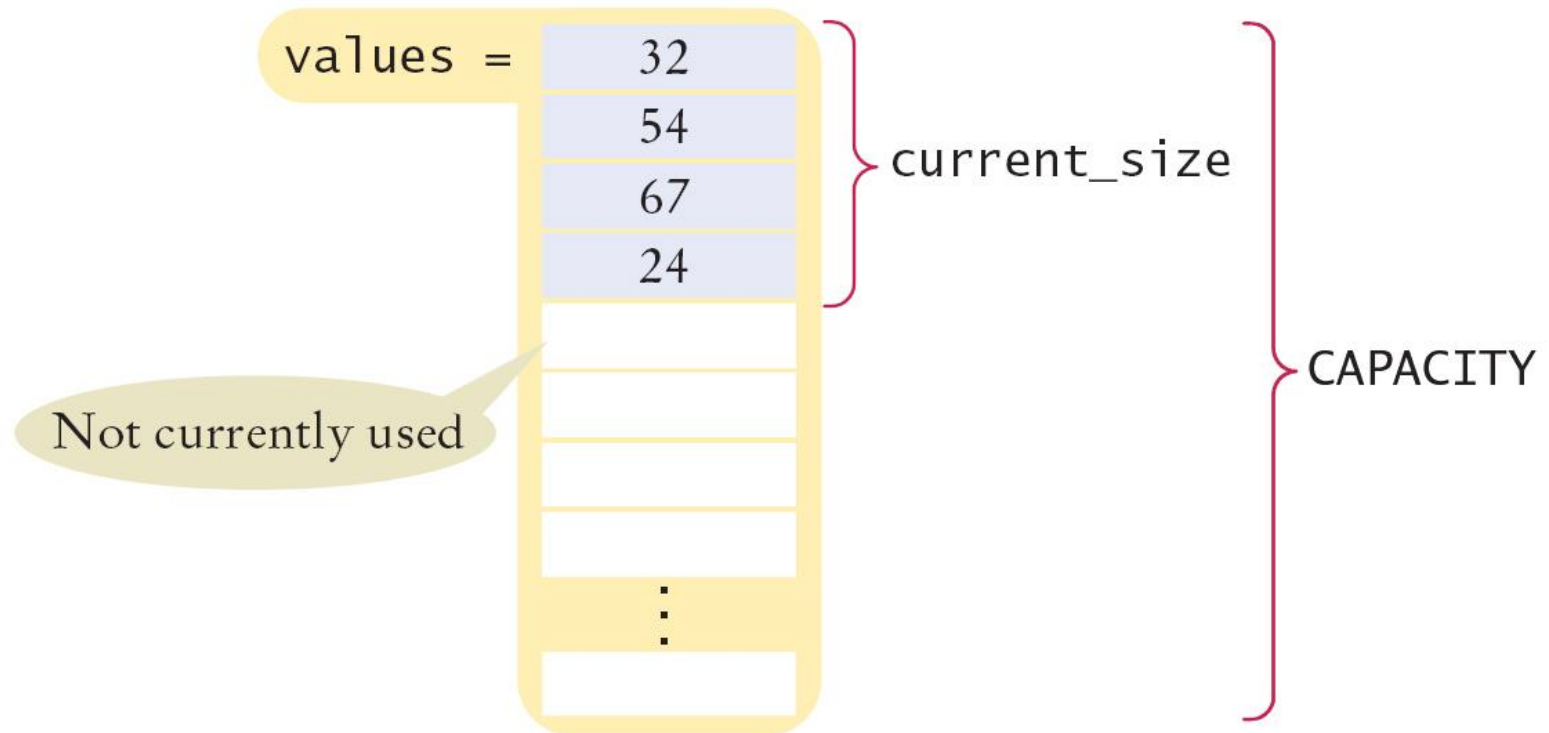
```
current_size = 4; // array now holds 4
```



Partially-Filled Arrays –Variable for Size

```
const int CAPACITY = 100;  
double values[CAPACITY];
```

```
current_size = 4; // array now holds 4
```



Partially-Filled Arrays – Capacity

- The following loop fills an array with user input.

```
const int CAPACITY = 100;  
double values[CAPACITY];
```

```
int size = 0;  
double input;  
while (cin >> input)  
{  
    if (size < CAPACITY)  
    {  
        values[size] = x;  
        size++;  
    }  
}
```



- *Each time the size of the array changes we update this variable:*

Partially-Filled Arrays – Capacity

- The following loop fills an array with user input.

```
const int CAPACITY = 100;
double values[CAPACITY];

int size = 0;
double input;
while (cin >> input)
{
    if (size < CAPACITY)
    {
        values[size] = x;
        size++;
    }
}
```

- *Each time the size of the array changes we update this variable:*
- When the loop ends, the companion variable `size` has the number of elements in the array

Partially-Filled Arrays – Capacity

How would you print the elements in a partially filled array?

By using the `current_size` companion variable.



```
for (int i = 0; i < current_size; i++)  
{  
    cout << values[i] << endl;  
}
```

Partially-Filled Arrays – Capacity

To visit all elements of an array, use a variable for the index.

A for loop's variable is best:



```
for (int i = 0; i < CAPACITY; i++)  
{  
    cout << values[i] << endl;  
}
```

- When *i* is 0, `values[i]` is `values[0]`, the first element.
- When *i* is 1, `values[i]` is `values[1]`, the second element

Partially-Filled Arrays – Capacity

To visit all elements of an array, use a variable for the index.

A for loop's variable is best:



```
for (int i = 0; i < CAPACITY; i++)  
{  
    cout << values[i] << endl;  
}
```

- When *i* is 0, values[*i*] is values[0], the first element.
- When *i* is 1, values[*i*] is values[1], the second element
- ...
- When *i* is 9, values[*i*] is values[9], the *last legal* element.

Partially-Filled Arrays – Capacity

To visit all elements of an array, use a variable for the index.

A for loop's variable is best:

```
for (int i = 0; i < CAPACITY; i++)  
{  
    cout << values[i] << endl;  
}
```

Note that the loop condition is that the index is *less than* **CAPACITY**

Because there is no element corresponding to `values[10]`.

Even though 10 (CAPACITY) is the number of elements we want to visit

Illegally Accessing an Array Element

A *bounds* error occurs when you access an element outside the legal set of indices:

```
cout << values[10];
```

Doing this can corrupt data
or cause your program to terminate.

DANGER!!!

DANGER!!!

DANGER!!!

Use Arrays for Sequences of Related Values

Recall that the type of every element must be the same.

That implies that the “meaning” of each stored value is the same

```
int scores[NUMBER_OF_SCORES];
```

Clearly the meaning of each element is a score.

Use Arrays for Sequences of Related Values

But an array could be used improperly:

```
double personal_data[3];  
personal_data[0] = age;  
personal_data[1] = bank_account;  
personal_data[2] = shoe_size;
```

Clearly these doubles do *not* have the same meaning!

Use Arrays for Sequences of Related Values

But worse:

```
personal_data[    ] = new_shoe_size;
```

Use Arrays for Sequences of Related Values

But worse:

```
personal_data[ ? ] = new_shoe_size;
```

Oh dear!

Which position was I using for the shoe size?

Use Arrays for Sequences of Related Values

Arrays should be used when
the meaning of each element is the same.